СВЯЗИ между таблицами

- users (teacher) 1 → M courses
- users (student) M → M courses через enrollments
- courses 1 → M lessons
- lessons 1 → M materials
- lessons 1 → M tests
- tests 1 → M questions
- questions 1 → M answer_options
- users (student) 1 → M attempts
- attempts 1 → M student_answers
- questions 1 → 1 student_answers (в рамках попытки)

ПЛАН ВКР

PostgreSQL + HTTP/REST + Qt

📋 Шаг 9️⃣ ПОДРОБНЫЙ ПЛАН ВКР (итог)

Теперь давай зафиксируем **окончательный план**, который можно нести научнику:

Глава 1. Анализ предметной области
- дистанционное обучение
- требования к LMS
- постановка задачи

Глава 2. Проектирование системы
- архитектура клиент–сервер
- роли пользователей
- модель данных (словесная + ER)
- сценарии использования

Глава 3. Реализация программного обеспечения
- выбор технологий
- серверная часть
- клиентское приложение
- реализация тестирования

Глава 4. Тестирование и анализ результатов
- стратегия тестирования
- модульные тесты
- интеграционные сценарии

🔷 Этап 1. Анализ и проектирование

(мы его почти завершили)
- постановка задачи
- роли пользователей
- архитектура клиент–сервер
- модель данных

📌 Результат:
описание системы + ER-модель БД

🔷 Этап 2. Реализация серверной части (C++)

Подэтапы:
- HTTP-сервер
- авторизация и роли
- работа с PostgreSQL
- логика тестирования
- расчёт результатов

✅ Сервер = **ядро системы**

🔷 Этап 3. Реализация клиентского приложения (Qt)

Подэтапы:
- окно входа
- список курсов
- просмотр уроков
- прохождение тестов
- просмотр результатов

📌 UI — только необходимый минимум (MVP).

🔷 Этап 4. Интеграция и тестирование

- проверка клиент–серверного взаимодействия
- тестирование сценариев
- обработка ошибок

🔷 Этап 5. Анализ результатов и оформление ВКР

- описание реализации
- скриншоты
- выводы

Шаг 1️⃣ Архитектура ПО (уровень модулей)

Теперь мы можем разложить систему на модули.

🖥 **Сервер (C++)**
- HTTP Server
- Auth Module
- Course Module
- Test Module
- Result Module
- DB Access Layer

🔁 Формулировка:
«Серверная часть реализована в виде набора модулей, каждый из которых отвечает за отдельную функциональную область».

🖥 **Клиент (Qt)**
- UI (окна, формы)
- Network Layer (HTTP-клиент)
- Models (данные)
- Controllers (логика)

👉 Можно красиво сослаться на **MVC / MVVM**.

Шаг 1️⃣ Общая структура серверного приложения

Представь сервер как **набор слоёв**, а не как «main.cpp на 2000 строк».

```
Server
├── HTTP layer        (приём запросов)
├── Controller layer  (обработка API)
├── Service layer     (бизнес-логика)
├── Data Access       (работа с БД)
└── Models            (структуры данных)
```

🧠 Формулировка для ВКР:
«Серверная часть реализована с разделением ответственности между слоями, что повышает поддерживаемость и расширяемость системы».

Шаг 1️⃣ Модули сервера (что именно будет)

🔷 **1. HTTP layer**

Отвечает за:
- запуск HTTP-сервера
- приём запросов
- отправку ответов

📌 Здесь **нет логики обучения**, только транспорт.

🔷 **2. Controllers**

Отвечает за:
- разбор запроса
- проверку входных данных
- вызов сервисов

Примеры:
- AuthController
- CourseController
- TestController

📌 Один контроллер = одна группа API.

🔷 **3. Services (самое важное)**

Здесь живёт логика системы:
- проверка ролей
- подсчёт баллов
- правила прохождения тестов

Примеры:
- AuthService
- CourseService
- TestService

👉 Именно этот слой ты будешь описывать в ВКР подробнее всего.

🔷 **4. Data Access Layer (DAL)**

Отвечает за:
- SQL-запросы
- работу с PostgreSQL
- маппинг данных

Примеры:
- UserRepository
- CourseRepository
- TestRepository

📌 UI и сервисы **не знают SQL** — это большой плюс.

🔷 **5. Models**

Простые структуры:
- User
- Course
- Lesson
- Test
- Question

Без логики, только данные.

Шаг 1️⃣ Проектирование REST-API (логически, без кода)

Мы ЕЩЁ пишем код, а описываем контракты между клиентом и сервером.
Это огромный плюс на защите.

📘 **Авторизация и пользователи**

🔐 **Авторизация**
- `POST /api/auth/login`
  - → логин + пароль
  - ← токен / id пользователя / роль

📌 Используем простейшую сессию или токен, без OAuth (для ВКР достаточно).

👤 **Пользователь (чтение роли)**
- `GET /api/users/profile`
  - ← данные пользователя

📚 **Курсы**

📚 **Получение списка курсов**
- `GET /api/courses`
  - студент → доступные курсы
  - преподаватель → свои курсы

📘 **Детали курса**
- `GET /api/courses/{courseId}`
  - ← описание, уроки

✨ **Создание курса (преподаватель)**
- `POST /api/courses`

📚 **Уроки и материалы**

📖 **Уроки курса**
- `GET /api/courses/{courseId}/lessons`

📎 **Материалы урока**
- `GET /api/lessons/{lessonId}/materials`

⬆️ **Загрузка материала (преподаватель)**
- `POST /api/materials`

📌 Файл загружается + сохраняется путь в БД.

📝 **Тестирование (ядро проекта)**

📝 **Получение теста**
- `GET /api/tests/{testId}`
  - ← вопросы (без правильных ответов)

▶️ **Начало попытки**
- `POST /api/tests/{testId}/start`
  - ← `attemptId`

📤 **Отправка ответов**
- `POST /api/attempts/{attemptId}/submit`
  - → ответы
  - ← результат, балл

📊 **Результаты и успеваемость**

📊 **Результаты студента**
- `GET /api/results/my`

📊 **Статистика курса (преподаватель)**
- `GET /api/courses/{courseId}/stats`

🔍 **Контрольная точка (один вопрос)**

Ответь честно:
> Нужно ли **хранить правильные ответы** в ответе `GET /api/tests/{testId}`?

- ☐ да
- ☐ нет

(Это очень важный момент безопасности и архитектуры.)

АРХИТЕКТУРА c++ КЛАССОВ СЕВРЕРА

1️⃣ Базовая идея (зафиксируй в голове)

👉 **Один класс = одна ответственность**
- контроллеры — принимают HTTP
- сервисы — думают
- репозитории — работают с БД
- модели — просто данные

2️⃣ Models (самые простые)

Это **структуры данных**, без логики.

Примеры (логически, не код):
- `User`
- `Course`
- `Lesson`
- `Test`
- `Question`
- `Attempt`

📌 Правило:
Models не знают ни про HTTP, ни про БД.

3️⃣ Repository layer (доступ к PostgreSQL)

Каждый репозиторий работает с **одной сущностью**.

Примеры:
- `UserRepository`
- `CourseRepository`
- `TestRepository`
- `AttemptRepository`

Отвечает за:
- SQL-запросы
- преобразование `PGresult → Model`

📌 Репозиторий **не проверяет роли**, не считает баллы.

4️⃣ Service layer (САМОЕ ВАЖНОЕ)

Здесь вся логика системы.

🔷 **AuthService**

Отвечает за:
- проверку логина/пароля
- определение роли
- авторизацию

🔷 **CourseService**

Отвечает за:
- доступ к курсам
- проверку прав (студент/преподаватель)
- получение структуры курса

🔷 **TestService**

Отвечает за:
- выдачу теста (без правильных ответов)
- старт попытки
- проверку ответов
- подсчёт баллов

📌 **Именно TestService ты будешь подробно описывать в ВКР.**

5️⃣ Controller layer (HTTP)

Контроллер = «переводчик» между HTTP и сервисами.

Примеры:
- `AuthController`
- `CourseController`
- `TestController`

Контроллер:
- читает JSON
- вызывает сервис
- возвращает JSON

❌ Не считает баллы
❌ Не пишет SQL

6️⃣ Связь между слоями (очень важно)

Логика такая:

```
HTTP Request
     ↓
Controller
     ↓
Service
     ↓
Repository
     ↓
PostgreSQL
```

Обратный путь — вверх.

🗣 Формулировка для ВКР:
> «Взаимодействие между компонентами осуществляется последовательно, что исключает жёсткие зависимости между слоями».

7️⃣ Ответственность `AuthService`

`AuthService` отвечает за:
- вход пользователя
- регистрацию пользователя
- проверку паролей
- определение роли

❌ Он не:
- парсит HTTP
- работает с JSON
- пишет SQL

8️⃣ Зависимости `AuthService`

Логически:

```
AuthService
     ↓
UserRepository
```

👉 Это значит:
`AuthService` получает `UserRepository` через конструктор
(Dependency Injection — можно упомянуть в ВКР).

Шаг 2️⃣ Вводим отдельный компонент: `PasswordHasher`

Теперь архитектура становится **зрелой**.

1️⃣ Назначение `PasswordHasher`

Отвечает **ТОЛЬКО** за:
- хэширование пароля
- проверку пароля по хэшу

AuthService **не знает**, как это делается.

2️⃣ Логическая схема теперь такая

```
AuthController
     ↓
AuthService
     ↓      ↓
UserRepo   PasswordHasher
     ↓
PostgreSQL
```

📌 Это уже архитектура уровня «настоящий backend».

3️⃣ Интерфейс `PasswordHasher` (логически)

Без реализации, только контракт:
- `hash(password) → string`
- `verify(password, hash) → bool`

📌 Реализация:
- сначала можно сделать **простейшую (например SHA-256)**
- в тексте ВКР напишешь: *«алгоритм может быть заменён без изменения бизнес-логики»*

1️⃣ Какие методы будут у `AuthService`

Предлагаю **минимальный, но достаточный набор**.

🔷 **Вход (login)**

```
login(login, password) → AuthResult
```

🔷 **Регистрация (register)**

```
register(login, password, role) → AuthResult
```

📌 Роль:
- по умолчанию — `student`
- `teacher` / `admin` → только через админа (можно описать как ограничение)

2️⃣ Что такое `AuthResult`

Это **простая модель результата**, а не JSON.

Содержит:
- `success`
- `userId`
- `role`
- `errorMessage` (если ошибка)

📌 Это потом легко сериализуется в JSON в Controller.

3️⃣ Параллельно проектируем `UserRepository`

Теперь без кода — **что он делает**.

`UserRepository` отвечает за:
- поиск пользователя по логину
- создание нового пользователя
- получение роли и id

Методы (логически):
- `findByLogin(login)`
- `createUser(login, passwordHash, role)`
- `exists(login)`

4️⃣ Как будет выглядеть поток LOGIN (мысленно)

1. Controller получает HTTP-запрос
2. Controller вызывает `AuthService::login()`
3. `AuthService`
   - спрашивает `UserRepository`
   - проверяет пароль
   - формирует `AuthResult`
4. Controller превращает `AuthResult` в JSON

👉 **Controller не думает, Service думает.**

📁 src/models

Это **структуры данных** (без логики)
- `User` — пользователь
- `Course` — курс
- `AuthResult` — результат логина

👉 Models = просто данные

📁 src/controllers

Это слой **HTTP**
- `AuthController`
- `CourseController`

Они:
- принимают HTTP-запрос
- читают JSON или заголовки
- вызывают Service
- возвращают JSON

❌ Они не содержат бизнес-логики.

📁 src/services

Это **бизнес-логика**
- `AuthService`
- `CourseService`

Здесь:
- проверяются роли
- принимаются решения
- выполняются правила системы

👉 Это «мозг» системы.

📁 src/repositories

Это слой **доступа к данным**
- `UserRepository`
- `CourseRepository`
- `EnrollmentRepository`
- InMemory версии

Они:
- хранят данные
- возвращают данные
- не принимают решений

👉 Это «хранилище».

3️⃣ Как проходит один запрос (пример)

Допустим:

```
GET /api/courses
```

Что происходит:

1️⃣ CourseController
2️⃣ Передаёт userId и role в CourseService
3️⃣ CourseService решает:
- если Student → взять enrollments
- если Teacher → взять свои курсы
  - 4️⃣ Repository отдаёт данные
  - 5️⃣ Controller превращает их в JSON

4. Что такое Enrollment

Enrollment = запись студента на курс.

Это связь:

```
Student ← Course
```

В памяти мы хранили:

```
vector<pair<studentId, courseId>>
```

Позже это будет таблица в PostgreSQL.